

150. Ta fonction passe avant celle de la console

Si ton programme écrit une fonction du même nom qu'une fonction de la console, c'est la tienne qui compte.

1. Ce qu'il faut comprendre

Le seul changement : `outils.cpp` écrit de nouveau une `bande()`, mais **en pointillés** : une case sur deux. `#include <bande>` est toujours là, dans `principal.cpp`.

Laquelle gagne ? La tienne. Le compilateur ajoute une fonction de la console **seulement si le programme n'en a pas écrit une du même nom**. Ici, il voit ta `bande()` : il n'ajoute pas la sienne, et la ligne `#include <bande>` ne grave rien.

La boucle en pointillés : `i = i + 2` au lieu de `i++`. Déroulé de `bande(2, 5, ALPHABET[0], 10)` : `i = 0 → (2, 5)` ; `i = 2 → (4, 5)` ; `i = 4 → (6, 5)` ; `i = 6 → (8, 5)` ; `i = 8 → (10, 5)` ; `i = 10` : `10 < 10` est faux, fin. Cinq A, un trou entre chaque.

À quoi ça sert : à **essayer une autre version** d'une fonction de la console sans toucher au compilateur. Si la tienne te plaît, tu sais maintenant comment la faire entrer dans la console (étape précédente).

Essaie : supprime la `bande()` d'`outils.cpp` : celle de la console revient, et les bandes redeviennent pleines.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <bande>` → [148.1. La fonction bande\(\) — la même tuile, plusieurs fois](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1011 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
bande()	39 o	← main() — une fonction fournie par le langage
pile()	42 o	← main() — ta fonction
main()	83 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 2      38 o sur place, dans bande(), pile() + AdresseCase (10 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
bande() × 3      57 o sur place, dans main()
pile()  × 1      19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le changement est dans outils.cpp : il écrit sa propre bande().
// principal.cpp garde « #include <bande> », mais c'est la mienne qui sert.
```

```
#include <ALPHABET>
#include <bande>      // la bande() de la console... qui ne servira pas ici
#include "outils.cpp" // MA bande() (en pointillés) et pile()
```

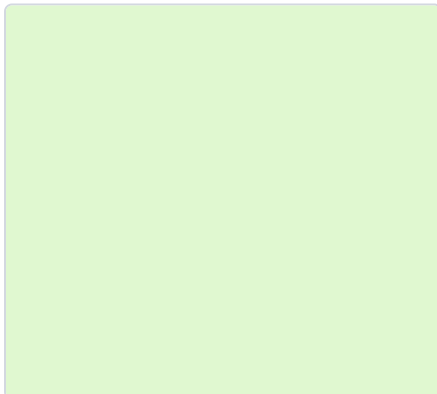
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    bande(2, 5, ALPHABET[0], 10); // MA bande() : cinq A, un trou entre chaque
    bande(2, 7, ALPHABET[1], 6);  // trois B : colonnes 2, 4, 6
    bande(2, 9, ALPHABET[2], 3);  // deux C : colonnes 2, 4
    pile(15, 4, ALPHABET[3], 6);

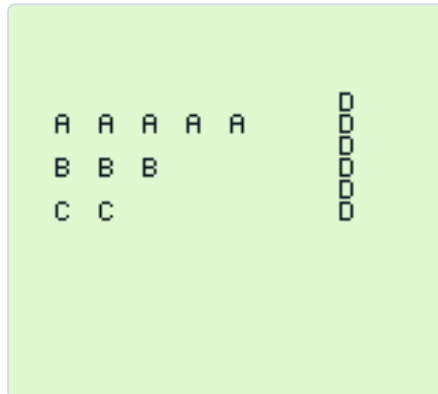
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Les bandes sont en pointillés : A A A A A, B B B, C C. La pile de D ne change pas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `ta bande()` est en pointillés : un A, un trou, un A... (A A A A)

✓ trois B, deux C, eux aussi en pointillés

✓ `pile()` n'a pas changé

Ensuite : **151. À toi : ajouter ta propre fonction de la console, pas à pas** — chapitre 25 : Tes propres `#include` : ajouter une fonction. · `gameboy3` — écrire une cartouche Game Boy en C++